M5：扩展宿主与 RPC 网关

注册事务、钩子链和外部控制合同

M5：扩展宿主与 RPC 网关
- 背景、目标与范围
- 扩展行为规格
  - M5-G-S01：发现与加载
  - M5-G-S02：工具名冲突
  - M5-G-S03：钩子责任链
  - M5-G-S04：reload 与卸载
- RPC 行为规格
  - M5-G-S05：传输与协议版本
  - M5-G-S06：命令与回应
  - M5-G-S07：事件流与关联
  - M5-G-S08：状态快照
  - M5-G-S09：背压与关闭
- 详细设计与数据结构
  - 注册事务与责任链
  - 网关与核心的依赖倒置
  - 后台任务和竞态
- 协议示例
- 验收矩阵
- 命令分发和扩展失败推演
  - 接受与完成分开
  - 注册失败只撤销宿主管理项
  - RPC模式的输出所有权
  - 关闭次序
- 备选方案、风险和实施顺序

M5：扩展宿主与 RPC 网关

属性	内容
状态	设计提案 1.0
前置	Core、Session、MCP
交付	`@local/gateway`、`--rpc`
验收	M5-G01 至 M5-G12

背景、目标与范围

扩展需要注册工具和拦截调用；编辑器或脚本需要通过进程协议提交任务并接收事件。这两种接入应使用同一 Session，避免产生重复工具循环或重复日志写入。

目标是可诊断、可撤销的扩展注册，以及不会被长运行 prompt 阻塞的 JSONL 控制通道。非目标包括远程 HTTP 服务、鉴权、多租户、热替换正在执行的扩展，以及任意第三方代码的安全隔离。加载本地 JavaScript 扩展就是执行宿主用户权限的代码。

扩展行为规格

M5-G-S01：发现与加载

用户扩展位于 `<agentHome>/extensions/*.mjs`；可信项目扩展位于 `<workspace>/.terminal-agent/extensions/*.mjs`。只取一层普通文件，不跟随 link；不运行 TypeScript 转译，不隐式执行 package.json scripts。按用户目录、项目目录，再按文件名码点顺序加载。

每文件默认导出 async/sync factory，签名 `(api:ExtensionAPI)=>void|Promise<void>`。factory timeout=5000ms；超时标记失败，不提交注册。由于 JS 不能中断同步死循环，这个 timeout 只是协作式 Promise 超时，不提供隔离；需要可靠隔离时另用 worker/进程设计。

factory 拿到一个暂存注册器，在 resolve 前的 registerTool/onBefore/onAfter 全部先存批次。只有成功返回且全部校验通过才整体提交。抛错、重复名、无效 schema 或超时丢弃该文件所有注册，诊断含 extensionId/path/code，后续扩展继续加载。

factory settle 后注册器关闭；延迟回调再 register 必须 INVALID_INPUT，不能在运行时偷偷改工具表。扩展顶层自行产生的外部副作用无法回滚，注册事务仅覆盖宿主管理的注册项。

M5-G-S02：工具名冲突

先注册内建工具，再注册已连接 MCP 工具，最后扩展。扩展不得覆盖已有名称；同一扩展中重复也使整批失败。MCP 名称生成已做确定性去冲突，扩展作者看到的是最终名称表。未加载项目扩展不能因同名文件被猜测存在而产生诊断，未信任时不读取它们。

注册后的每个工具记录 owner=`builtin|mcp:<server>|extension:<id>`，便于错误定位与卸载。工具声明快照与资源 prompt 统一发布；不能扩展提交一半后才发现 Session BUSY。

M5-G-S03：钩子责任链

before 钩子按扩展加载序、同文件注册序 await。输入为不可变 `{runId,callId,toolName,args}`，返回 void 或 `{block:true,reason?,terminate?}`。第一个 block 短路后续 before，生成 TOOL_BLOCKED。before 不允许修改参数；若需转换参数，应注册自己的包装工具。

after 钩子仅在实际 execute 结束后运行，输入含当前 outcome。每个返回部分 patch，逐个合并，后注册者对同字段覆盖。content、isError、details、terminate 可改；callId/toolName 不可改。任一 after 抛错转为 TOOL_FAILED 并停止该调用后续 after，不重复 execute。

每个钩子默认 5000ms，收到同一取消 signal。before timeout 按拒绝处理；after timeout 按失败处理。before 被拒绝的调用仍有 execution_end 和 tool result，不能留下未配对调用。

M5-G-S04：reload 与卸载

只在 Session idle 时 reload。创建候选资源快照、MCP 工具和扩展注册表，完整校验后由 CLI 一次发布工具表和 context。失败保留旧快照并诊断；新启动的候选 MCP 连接需关闭。扩展移除时撤销它的工具和钩子，但不删除旧消息。

动态 import 对同 URL 有缓存；1.0 reload 重新读取资源和 MCP 列表，扩展代码变更需重启进程，不宣称代码热更新。reload 返回结果应区分 `resourcesReloaded` 与 `extensionsRequireRestart`。不通过时间戳 query 绕过缓存无限累积模块。

RPC 行为规格

M5-G-S05：传输与协议版本

`--rpc` 启动一个 Session。stdin 每行一个 JSON 对象，stdout 每行一个响应或事件，均 UTF-8。协议 version=1。分帧采用 M5 MCP 的增量 UTF-8 与行上限规则，但 RPC 单个坏行返回 INVALID_INPUT 并继续读下一行，不关闭整个进程；超过 1 MiB 时丢弃至下个 LF 后重新同步。日志、启动提示和扩展 console 输出必须路由到 stderr；加载扩展前安装受控日志桥，宿主本身不得 console.log。

输入必须有非空 string id，长度<=128，进程内唯一。已用 ID 保留到进程退出；重复返回 INVALID_INPUT，不覆盖原请求。无有效 id 的坏行用 id=null 返回，不能臆造一个 ID。最多10000个请求 ID，达到后返回 INVALID_INPUT，要求重启连接，避免集合无限增长。

M5-G-S06：命令与回应

所有输入有 `{version:1,id,command,...}`；未知字段拒绝，不能默默接受拼写错误。接受阶段立即有一次 response，除 prompt/compact 之外的短操作可等实际完成后回应。prompt 与 compact 的长任务通过事件报告结果，避免阻塞 abort。

command	字段	前置与行为	response.data
`prompt`	text	idle；接受后启动运行	`{accepted:true,runId}`
`continue`	无	idle、上下文合法	`{accepted:true,runId}`
`steer`	text	可 busy，入 steer 队列	`{queueId}`
`follow_up`	text	可 busy，入 follow-up	`{queueId}`
`abort`	无	任意；请求取消 run/compact，不等 idle	`{requested:true}`

command	字段	前置与行为	response.data
`get_state`	无	任意；只读快照	下述 RpcState
`clear_queue`	无	任意；清未消费项	`{steering:QueueItem[], followUp:QueueItem[]}`
`compact`	instructions?	idle；接受后异步压缩	`{accepted:true,operationId}`
`checkout`	entryId:string	null	idle；等待日志提交
`reload`	无	idle；等待快照提交	资源与重启状态

response=`{version:1,type:'response',id,command,ok:true,data}` 或 `ok:false,error:{code,message,details?}`。无效命令不改 Session。prompt 的接受响应必须先写入并通过背压队列，之后才开始该 run 的事件；为此 Session 提供“预留 runId 后启动”的 gateway 内部接口，或 Core 允许在 prompt 调用时传入由网关生成的 runId。该入口只分配标识，不创建第二套运行状态。

M5-G-S07：事件流与关联

RPC 事件信封为 `{version:1,type:'event',seq,requestId,event}`。seq 为整个进程单调递增，独立于 Core 的 run seq；requestId 指向启动 run/compact 的命令，不是最近的 get_state/abort。Core 事件保留内部 runId/seq/turnId，不能丢掉关联。

compact 发 operation_start/operation_end，含 operationId 和结果；失败也有 operation_end，error 字段给原因。一次已接受的 prompt/continue 有一个 agent_end（存储灾难异常则 gateway 发 operation_error 诊断事件并标 faulted），不能再额外发第二个 response。abort 的 response 只承诺已请求，不承诺工具已退出，观察 agent_end/operation_end 才知道停止。

入站处理循环不能 `await session.prompt()` 后才读取下一行。校验并原子预留运行槽，挂后台 Promise，立即恢复读。两个连续 prompt：第一个接受，第二个 BUSY。get_state 和 abort 在第一个未完成时仍有响应。

M5-G-S08：状态快照

```
type RpcState = {
  sessionId: string;
  phase: "idle" | "running" | "compacting" | "faulted" | "closed";
  runId: string | null;
  leafId: string | null;
  messageCount: number;
  queued: { steering: number; followUp: number };
  tools: { name: string; owner: string }[];
  model: { provider: string; id: string; contextWindow: number };
};
```

不返回全历史、环境变量、MCP env、密钥或未经清理的错误堆栈。需要历史时可未来新增分页命令，不把其塞入 get_state。

M5-G-S09：背压与关闭

stdout 使用单个有界写队列，写返回 false 时等 drain。message_update 和 tool_execution_update 可按 messageId/callId 合并为最新一条，最终 message_end、response 和终态事件不可丢。待写字节上限 4 MiB，超过时暂停读新命令；仍继续采集终态，若无法保持上限则请求取消并退出失败，stderr 报 slow consumer。禁止无限缓存输出。

stdin EOF：不再接受命令，取消活动任务，等 Session idle、事件队列 flush、关闭 MCP、Session.close，再退出。stdout EPIPE：停止写，取消任务并清理后退出 1，不能对已经断开的管道无限重试。SIGINT 同样走统一清理入口。多个关闭信号只触发一次清理。

详细设计与数据结构

```
interface ExtensionAPI {
  registerTool(tool: AgentTool): void;
  onBefore(hook: BeforeHook): void;
  onAfter(hook: AfterHook): void;
  log(level: "info" | "warn" | "error", message: string): void;
}
type RegistrationBatch = {
  extensionId: string;
  tools: AgentTool[];
  before: BeforeHook[];
  after: AfterHook[];
  closed: boolean;
};
type ToolRegistryEntry = {
  tool: AgentTool;
  owner: string;
};
type RpcCommand =
  | { command: "prompt" | "steer" | "follow_up"; text: string }
  | { command: "continue" | "abort" | "get_state" | "clear_queue" | "reload" }
  | { command: "compact"; instructions?: string }
  | { command: "checkout"; entryId: string | null };
type RpcRequest = { version: 1; id: string } & RpcCommand;
```

BeforeHook/AfterHook 的参数与返回类型对应 M1 的钩子合同，应在 Core 导出并由 Gateway 使用；注册器不再定义一套看似相同却不兼容的 tool outcome。

注册事务与责任链

RegistrationBatch 是工作单元模式：先暂存，再完整验证、原子发布。它只能撤销注册项，不能回滚 factory 自行执行的网络/文件副作用。钩子链采用 Chain of Responsibility：before 可短路，after 逐层装饰结果，顺序是语义的一部分，不能 Promise.all。

网关与核心的依赖倒置

RPC dispatcher 接收 SessionPort（prompt/continue/abort/state 等方法），不解析 JSONL 存档，不直接调用工具。测试用带 gate 的 FakeSessionPort 验证并发命令；至少一条集成测试用真实 Session 和假模型，防止只证明 mock 行为。

输入解析、schema 校验、调度、输出编帧分为四个边界。RPC parser 只认识 JSON，不知道 Agent；dispatcher 只认识命令，不碰 stdout；writer 保证顺序与背压。中间错误都转成统一 Failure。

后台任务和竞态

活动任务表保存 `{operationId,requestId,promise,cancel}`，最多一项。接受前预留槽，启动失败必须释放。abort 不清槽，等任务 finally 清理。closed 标志阻止 EOF 后重入。用“一个状态写入者”的调度思想处理请求，不为每个命令开互相争锁的独立 Session。

协议示例

```
{"version":1,"id":"p1","command":"prompt","text":"读 README"}
{"version":1,"id":"q1","command":"get_state"}
{"version":1,"id":"a1","command":"abort"}
```

输出的必要偏序：p1 的 accepted response 在 p1 所有事件前；q1/a1 可以穿插在事件间；a1 response 之后可能还有已完成工具结果；p1 最终 agent_end 标明 aborted。不能把 a1 response 当成进程已停止的证据。

验收矩阵

编号	场景	断言
M5-G01	factory 注册两个工具后抛错	两者均不发布；下个扩展可加载
M5-G02	重名/schema 无效/延迟注册	整批失败；已有表不变；迟到注册拒绝
M5-G03	两个 before，首个 block	第二不调用；execute=0；结果可配对
M5-G04	多个 after patch/抛错/timeout	顺序确定；失败不重执行工具
M5-G05	未信任项目和用户扩展	项目目录零读取；用户扩展按名排序
M5-G06	prompt 挂 gate，同时 get_state/abort	两个短命令有 response；不被 prompt await 阻塞
M5-G07	两个连续 prompt；重复 id	首个接受，其余 BUSY/INVALID_INPUT；无额外 run
M5-G08	畸形 JSON/超长行/有效下一行	错行单个失败；下一行能解析
M5-G09	协议关联与输出污染	accepted 在首事件前；requestId 正确；stdout 每行可解析
M5-G10	write=false，随后 drain	顺序保持；updates 可合并；终态和 response不丢
M5-G11	stdin EOF/EPIPE/SIGINT	一次清理；取消活动运行；锁与进程释放

编号	场景	断言
M5-G12	reload busy/失败/扩展代码变更	旧快照保留；明确报告需重启

命令分发和扩展失败推演

接受与完成分开

p1被接受分配runId=R，writer先排入accepted并等待其写入；此后才启动R。q1随后查询时可以看到running，a1请求abort立即返回requested。R的最终事件仍带requestId=p1，不改成a1。p1没有第二条response，客户端用agent_end判断完成。

如果accepted无法写入（EPIPE），不启动R，清理并退出；不能在客户端根本不知道任务被接受时继续改文件。若accepted写出后启动遇到存储错误，发operation_error并进入faulted，不能再声称p1未被接受。

注册失败只撤销宿主管理项

扩展E1先registerTool(A)、onBefore(H)，再抛错；A/H均不发布。扩展E2正常注册B；B可用。E1若在factory中自行写了文件，宿主不能回滚该文件。注册事务确保表完整，不保证任意代码副作用原子。

before链H1通过、H2拒绝、H3从未运行；after链不运行，因为execute没有开始。对于execute成功但after失败，日志应说明执行结果可能已发生，不能重试工具“修复钩子错误”。M1将原outcome放details便于诊断。

RPC模式的输出所有权

只有writer可写stdout。内建诊断、MCP stderr、扩展api.log全部走stderr。TUI不在rpc模式初始化，不能发送ANSI。测试逐行JSON.parse全部stdout，而不只检查最后一个response；否则启动banner等污染容易漏掉。

get_state是实时只读快照，可以在message_update期间返回，但不能包含可被调用者修改的messages引用。输出背压时合并updates只影响显示精度，message_end与终态始终保留完整文本；客户端不能把delta数量当文本长度依据。

关闭次序

stdin EOF后，dispatcher置closed防止新请求；请求取消；等待领域终态；flush不可丢事件；关闭MCP；Session.close释放锁；writer关闭。若stdout已经EPIPE，跳过flush但仍清理领域资源。各步骤错误收集到stderr，不因一个MCP关闭失败跳过终端/锁清理。

关闭必须有可配置总watchdog用于报告卡住的受信扩展；到期不能谎报清理完成。若工具或扩展永久阻塞，同进程基线只能让上层进程管理器强杀，恢复依赖M2日志。

备选方案、风险和实施顺序

按输入顺序 await 所有命令实现最短，但无法及时 abort，因此采用短响应与事件完成。给扩展直接暴露 Session 会让它绕过日志与状态机，所以 API 只允许注册、钩子和日志。JavaScript 同进程扩展可以绕过受控日志桥直接写 stdout；对任意恶意代码没有隔离保证，受信扩展应遵循 API，敌对扩展需要独立进程。

先实现暂存注册器和钩子链，再做纯 RPC parser/dispatcher，之后后台运行关联与背压，最后接进程级清理。扩展和 RPC 共享工具表与 Session，但测试应分开，避免一个失败遮住另一个。